

1 Flow, tracking và các ràng buộc riêng (Optical Flow, Tracking and Their Constraints)

PHỤ THUỘC C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video (ngân sách bộ nhớ); B/07-chia-du-lieu-va-danh-gia (rõ ràng dữ liệu, chọn metric); A/01-toan-toi-thieu (bình phương tối thiểu, điều kiện của ma trận).

TÓM TẮT

Mục này bàn ba thứ mà một kỹ sư SLAM đã sống cùng hằng ngày, chỉ dưới tên khác: **dòng quang** (*optical flow*) là bài toán tương ứng điểm, **bám nhiều mục tiêu** (*multi-object tracking*) là bài toán liên kết dữ liệu, và ràng buộc nhân quả là ràng buộc cứng của mọi hệ chạy thời gian thực. Đọc xong bạn ánh xạ được các kỹ thuật học sâu ở đây về đúng chỗ tương ứng trong pipeline hình học của mình. Bạn chọn được metric tracking không đánh lừa. Và bạn biết kiến trúc nào bị loại ngay từ đầu khi không được nhìn khung hình tương lai. Nếu chỉ nhớ một điều: trong cả flow lẫn tracking, chỗ hệ thống sống hay chết là bước *liên kết* chứ không phải bước phát hiện — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng chỗ đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **optical flow** — trường vector (u, v) cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. “Dày đặc” nghĩa là tính cho mọi pixel, khác với việc chỉ bám vài trăm đặc trưng.
- **gradient ảnh** I_x, I_y, I_t — tốc độ thay đổi độ sáng theo trục ngang, trục dọc và theo thời gian tại một pixel. Ba đại lượng này là toàn tập dữ liệu mà phương trình flow cổ điển có.
- **bài toán khẩu độ (aperture problem)** — khi chỉ nhìn qua một cửa sổ nhỏ, chỉ thành phần chuyển động vuông góc với cạnh là quan sát được; thành phần dọc theo cạnh hoàn toàn vô hình. Đây là thiếu thông tin thật, không phải lỗi thuật toán.
- **liên kết dữ liệu (data association)** — bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking đó là ghép hộp với track; trong SLAM đó là ghép đặc trưng với điểm bản đồ. Gán sai gây hậu quả nặng và khó rút lại.
- **track ID** — mã số mà hệ thống gán cho một đối tượng và phải giữ nguyên suốt thời gian nó xuất hiện. *ID switch* là khi mã số bị đánh tráo giữa hai đối tượng — lỗi liên kết điển hình.
- **bộ lọc Kalman** — bộ ước lượng đệ quy giữ vị trí và vận tốc cùng độ bất định của chúng; nó dự đoán vị trí khung tới rồi cập nhật khi có phép đo mới. Trong tracking, nó cung cấp dự đoán để ghép cặp.
- **thuật toán Hungary** — thuật toán giải bài toán gán tối ưu: cho ma trận chi phí giữa m dự đoán và n phát hiện, tìm phép ghép một-một có tổng chi phí nhỏ nhất.
- **MOTA và HOTA** — hai thước đo tracking. MOTA cộng số phát hiện sót, phát hiện thừa và số lần đổi mã rồi chia cho tổng số vật thật. HOTA tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA) rồi lấy trung bình nhân của chúng.
- **nhân quả / streaming** — ràng buộc rằng quyết định tại thời điểm t chỉ được dùng dữ liệu tới t , không được dùng khung hình tương lai. Nó loại bỏ mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước.

Đặt vấn đề

Hai mục trước xử lý video như một khối cần phân loại. Nhưng phần lớn ứng dụng thật lại cần thứ tinh hơn nhiều: *cùng một điểm vật lý ở khung này nằm ở đâu trong khung kia*. Trả lời câu đó cho mọi pixel là optical flow; trả lời cho từng vật thể qua thời gian là tracking; và trong cả hai trường hợp, khó khăn thật không nằm ở việc nhìn thấy mà ở việc *ghép cặp*. Đây chính xác là cấu trúc của bài toán mà một hệ SLAM giải mỗi khung hình, nên mục này ít giới thiệu khái niệm mới hơn là dịch từ vựng.

Thêm vào đó là một ràng buộc mà ảnh tĩnh không có: mô hình chạy trực tuyến không được nhìn khung hình tương lai, và cũng không được nhìn quá lâu vào quá khứ nếu ngân sách mili-giây có hạn. Ràng buộc này loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng — đúng nghĩa **ràng buộc cứng** ở B/03-giai-phau-he-thong.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper flow hay tracking và biết ngay nó thay khối nào trong pipeline hình học quen thuộc, nó mua độ bền bằng giả định gì, và nó có chạy được dưới ràng buộc nhân quả của bạn hay không.

TRỰC GIÁC

Nhìn một cạnh thẳng đang chuyển động qua một cái ống hút: bạn chỉ thấy được thành phần chuyển động *vuông góc* với cạnh, còn thành phần dọc theo cạnh thì hoàn toàn vô hình. Đó không phải hạn chế của mắt hay của thuật toán, mà là thiếu thông tin thật sự: một phương trình, hai ẩn. Mọi thứ trong optical flow — lấy cửa sổ, làm trơn, kim tự tháp, khối tương quan — đều là các cách khác nhau để mượn thêm ràng buộc từ nơi khác. Và ma trận đo mức độ “mượn được hay không” chính là ma trận mà bạn đã dùng để chọn đặc trưng góc trong SLAM.

A. Optical flow: từ Lucas–Kanade tới RAFT

Giả định gốc là **độ sáng không đổi** (*brightness constancy*): cùng một điểm vật lý giữ nguyên giá trị sáng giữa hai khung. Viết $I(x, y, t) = I(x + u, y + v, t + 1)$ rồi khai triển Taylor bậc nhất cho phương trình ràng buộc

$$I_x u + I_y v + I_t = 0.$$

Câu này nói gì bằng lời: *tốc độ thay đổi độ sáng theo thời gian tại một pixel phải bằng đúng phần độ sáng bị “trôi vào” do chuyển động*. Một phương trình, hai ẩn — đó là **bài toán khẩu độ** (*aperture problem*) ở dạng đại số.

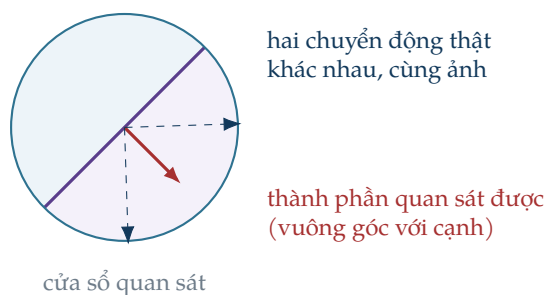
Lucas–Kanade mượn ràng buộc bằng cách giả định flow không đổi trong một cửa sổ nhỏ, cho N phương trình và hai ẩn, rồi giải bình phương tối thiểu. Phương trình chuẩn có ma trận hệ số

$$A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix},$$

và đây đúng là ma trận cấu trúc trong tiêu chuẩn chọn đặc trưng góc. Ý nghĩa: bài toán giải được khi và chỉ khi cửa sổ có gradient theo *hai* hướng độc lập. Ví dụ nhỏ nhất: một cửa sổ nằm trên một cạnh dọc lý tưởng có $I_x = g$ và $I_y = 0$ ở mọi pixel, nên $A^T A = \begin{bmatrix} Ng^2 & 0 \\ 0 & 0 \end{bmatrix}$ — suy biến, giá trị riêng nhỏ nhất bằng 0, và thành phần flow dọc theo cạnh không xác định.

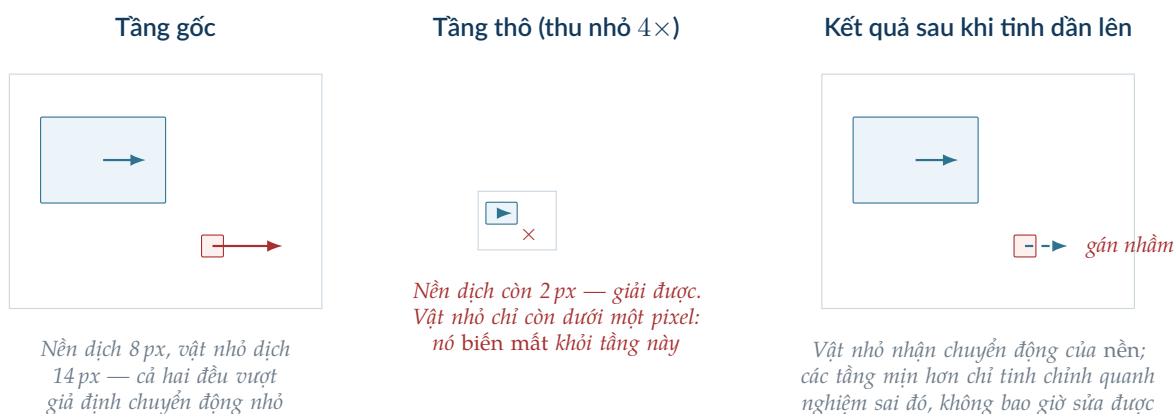
↪ **tiền đề** A/01 — đây đúng là câu hỏi điều kiện của một hệ bình phương tối thiểu: nghiệm ổn định khi giá trị riêng nhỏ nhất đủ xa 0, và suy biến khi nó bằng 0.

Chọn “good features to track” chính là chọn các cửa sổ mà giá trị riêng nhỏ nhất của ma trận này đủ lớn; nói cách khác, việc chọn đặc trưng trong SLAM và tính giải được của flow là *cùng một điều kiện*.



Hình 1. Bài toán khâu độ. Hai chuyển động thật rất khác nhau (mũi tên đứt) tạo ra cùng một hình ảnh bên trong cửa sổ, vì chỉ thành phần vuông góc với cạnh là quan sát được. Về đại số, đó là việc ma trận $A^T A$ suy biến; về thực hành, đó là lý do mọi bộ bám đặc trưng đều phải chọn góc chứ không chọn cạnh.

Hình 1 cũng giải thích luôn giả định thứ ba của Lucas–Kanade — *chuyển động nhỏ* — vì khai triển Taylor bậc nhất chỉ đúng khi dịch chuyển nhỏ hơn cỡ một pixel gradient. Cách chữa cổ điển là kim tự tháp thô–tối–mịn: giảm ảnh xuống bốn lần thì chuyển động 8 pixel thành 2 pixel. Cái giá là một chế độ hồng đặc trưng: vật *nhỏ* và *nhạy* biến mất ở tầng thô, nên hệ thống ước lượng chuyển động của nền thay vì của vật, và sai lầm đó không bao giờ được sửa ở các tầng mịn hơn.



Hình 2. Chế độ hồng đặc trưng của chiến lược thô–tối–mịn. Thu nhỏ ảnh làm chuyển động lớn trở nên nhỏ — đó là mục đích — nhưng nó cũng làm vật nhỏ biến mất trước khi kịp được ước lượng. Vì mỗi tầng chỉ *tinh chỉnh* nghiệm của tầng thô hơn, một sai lầm ở tầng thô là sai lầm vĩnh viễn. Đây chính là lý do RAFT bỏ kim tự tháp và tính tương quan toàn cặp một lần.

Hình 2 nên đọc như một cảnh báo quen thuộc với người làm bám đặc trưng: mọi sơ đồ nhiều tầng chỉ tinh chỉnh chứ không sửa lại đều thừa hưởng đúng chế độ hồng này, kể cả bộ tối ưu bạn tự viết.

Bảng 1. Bốn thể hệ optical flow. Cột cuối là cột đáng giá nhất: mỗi phương pháp mượn ràng buộc từ một nguồn khác nhau, nên nó hỏng khi đúng nguồn đó cạn.

Phương pháp	Mượn ràng buộc từ đâu	Mạnh khi nào	Hỏng khi nào
Lucas–Kanade	Giả định flow hằng trong cửa sổ nhỏ	Chuyển động nhỏ, kết cấu giàu, rất rẻ	Vùng trơn, cạnh dài, chuyển động lớn; thay đổi độ sáng phá giả định gốc
LK + kim tự tháp	Thêm giả định trơn qua các tỉ lệ	Chuyển động lớn của vùng lớn	Vật nhỏ chuyển động nhanh biến mất ở tầng thô
FlowNet, PWC-Net	Học từ dữ liệu; PWC-Net đưa kim tự tháp, warping, khối chi phí thành kiến trúc [1], [2]	Cân bằng tốt giữa chính xác và tốc độ	Lệch miễn so với dữ liệu tổng hợp dùng để huấn luyện
RAFT	Khối tương quan <i>toàn cặp</i> + tinh chỉnh lặp bằng GRU [3]	Chuyển động lớn, biên sắc, khái quát tốt	Bộ nhớ của khối tương quan; số vòng lặp là ngân sách phải tự chọn

RAFT đáng chú ý riêng vì hai lý do rất gần với kinh nghiệm tối ưu phi tuyến. Thứ nhất, nó bỏ chiến lược thô–tối–mịn bằng cách tính tương quan giữa *mọi* cặp pixel một lần, nhờ đó chuyển động lớn không còn cần kim tự tháp. Cái giá đo được: ở độ phân giải $1/8$ của ảnh 1024×436 , lưới là $128 \times 54 \approx 6900$ điểm, nên khối toàn cặp có $6900^2 \approx 4,8 \times 10^7$ phần tử — khoảng 190 MB ở fp32, trước khi tính bất cứ thứ gì. Thứ hai, nó tinh chỉnh nghiệm bằng một số vòng lặp cố định thay vì giải một lần; điều này khiến *số vòng lặp trở thành nút vận độ trễ*, đúng như việc bạn giới hạn số bước Gauss–Newton trong một bộ tối ưu thời gian thực. Đây là chỗ nổi đáng nhớ: mô hình học sâu ở đây không thay thế tư duy tối ưu lặp, nó mượn lại tư duy đó.

B. Nhãn nằm trên khoảng thời gian, không nằm trên khung hình

Nhận dạng hành động trên clip đã cắt sẵn là bài toán dễ; *temporal action localization* mới là bài toán thật: cho một video dài, tìm *khoảng* nào chứa hành động gì. Việc nhãn nằm trên khoảng kéo theo hai hệ quả mà người quen bài toán ảnh dễ vấp.

- **Metric phải đổi.** Ta không đánh giá theo khung hình mà theo khoảng, với ngưỡng *temporal IoU*. Ranh giới hành động vốn mơ hồ ngay cả với người gán nhãn — “bắt đầu ngồi xuống” là khung nào? — nên độ chính xác ở ngưỡng tIoU chặt phản ánh sự bất đồng của người gán nhãn nhiều hơn phản ánh chất lượng mô hình.
- **Chia dữ liệu phải theo video, không theo clip.** Hai clip cắt từ cùng một video có nền, ánh sáng và diễn viên giống nhau; để chúng nằm hai bên tập train/test là rò rỉ dữ liệu kinh điển, và nó thổi phồng kết quả một cách rất khó phát hiện.

↔ **trường hợp riêng của B/07** — hai clip cắt từ cùng một video là một nhóm phụ thuộc; chia ngẫu nhiên theo clip là rò rỉ dữ liệu kinh điển, chỉ đổi trục từ không gian sang thời gian.

C. Tracking chính là liên kết dữ liệu

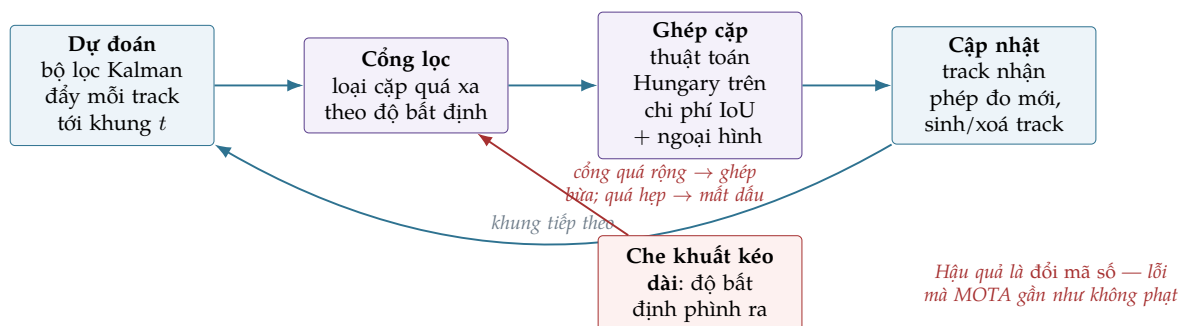
Có hai triết lý. *Tracking-by-detection* tách hẳn hai việc: một detector chạy độc lập trên mỗi khung, rồi một khối liên kết nối các phát hiện qua thời gian bằng một mô hình chuyển động cộng một độ đo tương đồng. *Joint detection-tracking* huấn luyện cả hai cùng lúc, mạnh hơn trên giấy nhưng khó gỡ lỗi vì không còn ranh giới rõ để đặt điểm dừng. Với hệ thống sản xuất, cách thứ nhất

thường thắng vì lý do kỹ thuật hệ thống chứ không phải vì độ chính xác: bạn thay được detector mà không phải huấn luyện lại phần liên kết.

Khối liên kết là chỗ mà người làm SLAM sẽ thấy quen tới mức bất ngờ. SORT dùng đúng bộ đôi kinh điển — bộ lọc Kalman dự đoán vị trí khung tới, rồi thuật toán Hungary ghép cặp dự đoán với phát hiện theo chi phí IoU [4]. ByteTrack thêm một quan sát đơn giản mà hiệu quả: đừng vứt các phát hiện điểm thấp, hãy dùng chúng ở vòng ghép thứ hai, vì một vật bị che một phần thường cho điểm thấp chứ không phải không có [5]. Về mặt cấu trúc, đây là cùng bài toán **liên kết dữ liệu** (*data association*) mà một hệ SLAM giải khi ghép đặc trưng với điểm bản đồ: có dự đoán từ mô hình chuyển động, có cổng lọc theo khoảng cách thống kê, có một bài toán gán tối ưu, và có hậu quả nghiêm trọng khi gán sai.

→ cùng nguyên lý với C/16 — ghép hộp với track và ghép đặc trưng với điểm bản đồ là cùng một bài toán gán tối ưu có cổng lọc theo độ bất định; chỉ khác hậu quả khi gán sai.

Khác biệt lớn nhất là ở SLAM, một liên kết sai làm hỏng bản đồ và rất khó rút lại, trong khi ở tracking nó chỉ đổi một mã số — nhưng chính vì hậu quả nhẹ hơn nên metric mới dễ che giấu nó.



Đây đúng vòng lặp mà một hệ SLAM chạy mỗi khung hình, chỉ khác tên gọi: dự đoán tư thế, giới hạn vùng tìm kiếm, ghép đặc trưng với điểm bản đồ, rồi cập nhật trạng thái.

Hình 3. Tracking là một vòng lặp liên kết dữ liệu, không phải một mạng nơ-ron. Ba trong bốn khối không chứa tham số học được nào; phần học được nằm ở detector cấp phép đo và ở đặc trưng ngoại hình dùng làm chi phí ghép. Vì thế cải thiện detector và cải thiện liên kết là hai việc khác nhau, và một thước đo gộp chúng vào một số sẽ che mất việc nào đang hỏng.

Hình 3 là lý do hộp cam bẫy ngay dưới đây quan trọng đến vậy: nếu vòng lặp này hỏng mà thước đo không phản ánh, bạn sẽ dành công sức tối ưu nhầm khối.

CẠM BÃY

MOTA cộng ba loại lỗi rồi chia cho tổng số vật thể thật: $MOTA = 1 - (FN + FP + IDSW) / GT$. Vấn đề nằm ở việc FN và FP được đếm *trên từng khung hình cho từng vật*, còn mỗi lần đổi mã số chỉ được đếm *một lần*. Ví dụ: 1000 vật–khung, 50 FN, 30 FP, 40 lần đổi mã cho $MOTA = 0,88$; cải thiện khối liên kết để chỉ còn 20 lần đổi mã — tức tốt lên gấp đôi — chỉ nâng MOTA lên 0,90. Nói cách khác, một hệ thống liên tục đánh tráo danh tính vẫn có thể khoe MOTA cao. HOTA sửa điều này bằng cách tách riêng chất lượng phát hiện và chất lượng liên kết, rồi lấy trung bình nhân $\sqrt{DetA \cdot AssA}$. Hai thành phần khi đó có trọng số ngang nhau, và không thành phần nào bù đắp được cho thành phần kia [6], [7].

D. Nhất quán thời gian và ràng buộc nhân quả

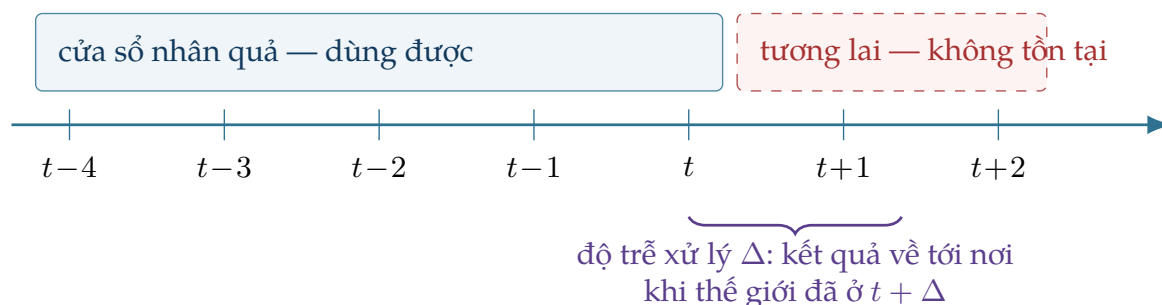
Một mô hình phân vùng chạy độc lập trên từng khung có thể đạt mIoU cao mà vẫn cho video nhấp nháy. Lý do là hàm mất mát không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau. Ba cách xử lý, với ba mức chi phí khác nhau:

Bảng 2. Ba cách chữa hiện tượng nhấp nháy, xếp theo mức xâm lấn vào hệ thống.

Cách	Cơ chế	Chi phí	Hỏng khi nào
Làm mượt hậu xử lý	Lọc dự đoán qua vài khung, có thể warp bằng flow	Rẻ, nhưng thêm độ trễ	Chuyển cảnh và chuyển động nhanh: làm mượt thành làm nhoè
Đưa thời gian vào mô hình	Mô hình nhận nhiều khung, hoặc mang trạng thái ẩn	Đắt nhất, chạm vào ngân sách ở Mục 2	Trạng thái ẩn có thể “dính” vào một sai lầm và tự củng cố
Loss nhất quán thời gian	Phạt sai khác giữa dự đoán của hai khung liên tiếp sau khi warp	Trung dung, chỉ tốn lúc huấn luyện	Warp dựa vào flow, nên flow sai thì loss dạy điều sai

↔ **ràng buộc bởi** C/15/02 — cách chữa “đưa thời gian vào mô hình” nghe hấp dẫn nhất nhưng chính là cách đắt nhất, vì nó tiêu vào đúng ngân sách ba chiều đã cạn ở mục trước.

Cả ba đều bị một ràng buộc lớn hơn chi phối: hệ thống chạy trực tuyến *không được nhìn khung hình tương lai*. Ràng buộc này loại thẳng mọi kiến trúc hai chiều, mọi phép làm mượt trung tâm, và mọi lịch trình đánh giá “xem toàn bộ video rồi mới quyết định” — và nó loại chúng trước khi độ chính xác được đem ra bàn.



Hình 4. Ràng buộc nhân quả, vẽ trên trục thời gian. Vùng xanh là toàn bộ thông tin mà một hệ streaming được phép dùng để quyết định tại thời điểm t ; vùng đỏ là thứ mà mọi mô hình hai chiều lạng lẽ giả định là có. Ngoặc tím nhắc thêm một điều thường bị quên: ngay cả quyết định đúng cũng đến muộn mất Δ , nên hệ thống hạ nguồn phải hoặc dự đoán bù, hoặc chấp nhận hành động theo một trạng thái đã cũ.

Hình 4 nói thêm một điều mà người làm robot cảm nhận rõ hơn ai hết: ràng buộc nhân quả không chỉ là “không được nhìn tương lai” mà còn là “kết quả luôn nói về quá khứ”. Vì thế, khi so sánh hai mô hình cho hệ thời gian thực, đại lượng đáng so không phải độ chính xác trên khung hình t . Đáng so là độ chính xác *tại thời điểm quyết định được dùng*. Cách đánh giá này vẫn còn ít được chuẩn hoá, và nhiều bảng kết quả công bố không nói gì về nó.

E. Giới hạn và trường hợp hỏng

- **Giả định độ sáng không đổi bị vi phạm thường xuyên hơn ta nghĩ.** Tự động phơi sáng, đèn nhấp nháy, phản xạ gương và bề mặt bóng đều làm sai giả định gốc của flow — và đây đúng là những cảnh làm hỏng cả các hệ thống hình học dựa trên độ sáng. Mô hình học sâu chịu

đựng tốt hơn nhưng không miễn nhiễm, vì dữ liệu huấn luyện của chúng phần lớn là ảnh tổng hợp phơi sáng cố định.

- **Che khuất kéo dài là kẻ thù của mọi bộ bám.** Mô hình chuyển động dự đoán được vài khung, sau đó độ bất định lớn tới mức cổng lọc hoặc quá rộng (ghép bù) hoặc quá hẹp (mất dấu). Không có cách chỉnh tham số nào thoát khỏi đánh đổi này; chỉ có thêm thông tin — ngoại hình, hoặc một cảm biến khác.
- **Metric đánh lừa theo hai kiểu.** MOTA che giấu lỗi liên kết như đã nói; còn mIoU theo khung che giấu hiện tượng nhấp nháy. Cả hai đều là ví dụ của cùng một sai lầm: chọn một con số tổng không đo được đại lượng mà hệ thống hạ nguồn thực sự cần.
- **Lệch miền của dữ liệu flow.** Vì nhãn flow dày đặc gần như không thể gán bằng tay, các mô hình được huấn luyện chủ yếu trên dữ liệu tổng hợp; chất lượng trên cảnh thật vì thế phụ thuộc vào mức độ giống nhau giữa hai miền, và điều đó thay đổi theo camera của bạn.

CẦN NHỚ

Trong flow cũng như trong tracking, phần khó không phải phát hiện mà là *liên kết* — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng phần đó. Hãy chọn metric tách riêng chất lượng phát hiện khỏi chất lượng liên kết (HOTA cho tracking, sai số theo từng chế độ chuyển động cho flow), rồi mới chọn mô hình. Ràng buộc nhân quả thì đứng trên tất cả: nó loại phương án trước khi độ chính xác kịp lên tiếng. [T2]

F. Kết nối

C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video là tiền đề về chi phí: mọi cách chữa nhấp nháy bằng “đưa thời gian vào mô hình” đều phải trả bằng ngân sách ba chiều ở đó. B/07-chia-du-lieu-va-danh-gia là nơi bàn tổng quát về việc chọn metric và về rõ ràng do nhóm phụ thuộc, mà chia dữ liệu theo clip là một ca điển hình. C/16-thi-giac-3d nối tiếp trực tiếp: flow là bài toán tương ứng hai chiều, còn chương 16 nâng nó lên tương ứng ba chiều và ràng buộc epipolar. C/14-bai-toan-cv-loi/04-bai-toan-lan-can chứa cấu trúc đánh đổi song song: top-down và bottom-up ở pose lặp lại đúng dạng của tracking-by-detection và joint tracking.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một phương trình độ sáng không đủ để giải ra flow, và ba giả định của Lucas–Kanade mượn ràng buộc còn thiếu từ đâu?
2. Viết ma trận $A^T A$ cho một cửa sổ nằm trên cạnh dọc lý tưởng và giải thích vì sao nó suy biến. Điều đó liên hệ thế nào với việc chọn đặc trưng góc?
3. Kim tự tháp thô–tối–mịn chứa được chuyển động lớn nhưng sinh ra chế độ hỏng nào, và nó biểu hiện ra sao trong một cảnh giao thông cụ thể?
4. RAFT bỏ kim tự tháp bằng khối tương quan toàn cặp. Cái giá tính được bằng con số là gì, và số vòng lặp đóng vai trò gì với một hệ thời gian thực?
5. Một hệ thống có MOTA 0,90 nhưng đổi mã số liên tục. Giải thích bằng công thức vì sao điều đó có thể xảy ra, và HOTA sửa nó thế nào?
6. Ràng buộc nhân quả loại bỏ những loại kiến trúc và loại đánh giá nào? Vì sao độ trễ Δ khiến “độ chính xác trên khung t ” là một đại lượng không đủ?
7. Bạn có mô hình phân vùng mIoU cao nhưng video nhấp nháy, và ngân sách tính toán đã cạn. Bạn chọn cách chữa nào trong ba cách, và chấp nhận hỏng ở đâu?

Nguồn

- [1] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [2] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [3] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [4] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [5] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [6] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [7] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.